Pruning eines neuronalen Netzwerks anhand von KI-Erklärbarkeit

Ilona Eisenbraun
Independent Coursework (ICW) 1
Master Angewandte Informatik
Fachbereich 4 - Informatik, Kommunikation und Wirtschaft
Hochschule für Technik und Wirtschaft Berlin, Deutschland
Ilona.Eisenbraun@Student.HTW-Berlin.de

I. Einleitung

Rund um das Pruning neuronaler Netze haben sich in den letzten Jahren eine Vielzahl an Forschungsarbeiten ergeben ([1], [2], [3]). Diese zielen vor Allem darauf ab, Modelle effizienter hinsichtlich des Speicherbedarfs, der Netzwerkgröße und der Berechnungszeit zu gestalten. Verschiedene Methodiken zeigen hier bereits immense Verbesserungen. In Anlehnung an meine in dem "Forschungsprojekt A" angefertigte theoretische Analyse zu den unterschiedlichen Pruning-Ansätzen widme ich mich in der nachfolgenden Ausarbeitung der Fragestellung, *wie* das Pruning praktisch umgesetzt werden kann. Spezifischer wird das Pruning anhand von KI-Erklärbarkeit (auch Explainable AI oder XAI genannt) untersucht. Da vor Allem für Außenstehende die Entscheidungsfindung eines Netzwerks - die beschreibt, *wieso* es eine gewisse Prädiktion fällt - undurchsichtig werden kann, zeigt XAI hierbei auf, *was* ausschlaggebend für etwa eine Klassifikation des Modells ist.

In diesem Rahmen werden verschiedene Pruning-Ansätze, die mit Erklärbarkeit in Verbindung stehen, auf einem Datensatz zur Hautkrebsklassifizierung gegen einander vergleichen. Diese Ausarbeitung dokumentiert dabei entstandene Lern-, Ergebnis- und Erkenntnisprozesse.

Hierfür leitet das Kapitel Grundlagen zuerst Konzepte des erklärbaren Pruning ein, um weiterleitend in Vorbereitung das Vorgehen für die nachfolgende Umsetzung zu beschreiben. Die resultierenden Ergebnisse werden in Evaluation beschrieben, um final im Kapitel Fazit eine Schlussfolgerung zu den in dieser Arbeit verwendeten Pruning-Methodiken zu schildern.

II. Grundlagen

Für die Umsetzung von Erklärbarkeits-basiertem Pruning ist es relevant, vorerst Hintergründe bedeutender Ansätze zu erläutern. Diesbezüglich gliedert sich dieses Kapitel in Erklärbare Relevanzen und Erklärbare Prototypen.

A. Erklärbare Relevanzen

Die "Relevance Scores" eines Netzwerks nutzend, zeigten Yeom et al. [4], wie Convolutional Neural Networks anhand dieser gepruned werden können. Dies geschieht über die "Layerwise Relevance Propagation" (LRP), welche ursprünglich der Domäne von Explainable AI zuzuordnen ist. Die Heatmap-Darstellung von LRP-Scores, angewandt auf die Dimensionen der Eingaben, visualisiert dabei üblicherweise Pixel, die am aussagekräftigsten für eine maschinelle Entscheidung sind.

LRP illustriert hierbei die Klassifikationsentscheidungen, indem der Output einer Prädiktion schrittweise durch die einzelnen Layer des Netzwerks zurück propagiert wird. Den einzelnen Einheiten (bzw. Gewichten) werden daraufhin Kontributionen zu dieser Entscheidung zugeordnet. Das Prinzip ähnelt einer Backpropagation von Gradienten mit dem Unterschied, Relevanzen der Einheiten auch über eine

Vielzahl an Layern zu konservieren, wodurch diese direkt mit dem Output eines Netzwerks in Verbindung gebracht und hinsichtlich ihrer Signifikanz bemessen werden können.

Für jedes Neuron i im Layer l wird hierbei die Relevanz als $R_i^{(l)}$ ermittelt, indem die Relevanz eines Neurons j aus dem nachfolgenden Layer (als $R_j^{(l+1)}$ beschrieben) von Output ausgehend berechnet wird. Zur Inferenz werden die gesamten Relevanzen summiert:

$$R_i^{(l)} = \sum_j \frac{(a_i^{(l)} w_{ij})^+}{\sum_{i'} (a_{i'}^{(l)} w_{i'j})^+} R_j^{(l+1)} \tag{1}$$

Die LRP-Regel führt für jedes Layer folgende zwei Schritte durchführt: Erstens werden die Aktivierungen der Einheiten aus der Propagierung ermittelt, die maßgebend zur Entscheidung beitragen. Diese sind in der Formulierung 1 im oberen Term der Summationsgleichung zu sehen. Zweitens werden auch die negativen Kontributionen bemessen, mit den Bestandteilen der LRP-Regel summiert und anhand der optionalen Hyperparameter α und β gewichtet (im unteren Term abgebildet).

Da LRP aussagekräftig für relevante Einheiten ist, kann es ebenso für das Ermitteln von belanglosen Parametern verwendet werden. Die sich über die Relevanz-Propagierung als irrelevant herausstellenden Einheiten des Netzwerks können durch die Bemessung gekappt werden. Die Abbildung 1 veranschaulicht diesen Prozess.

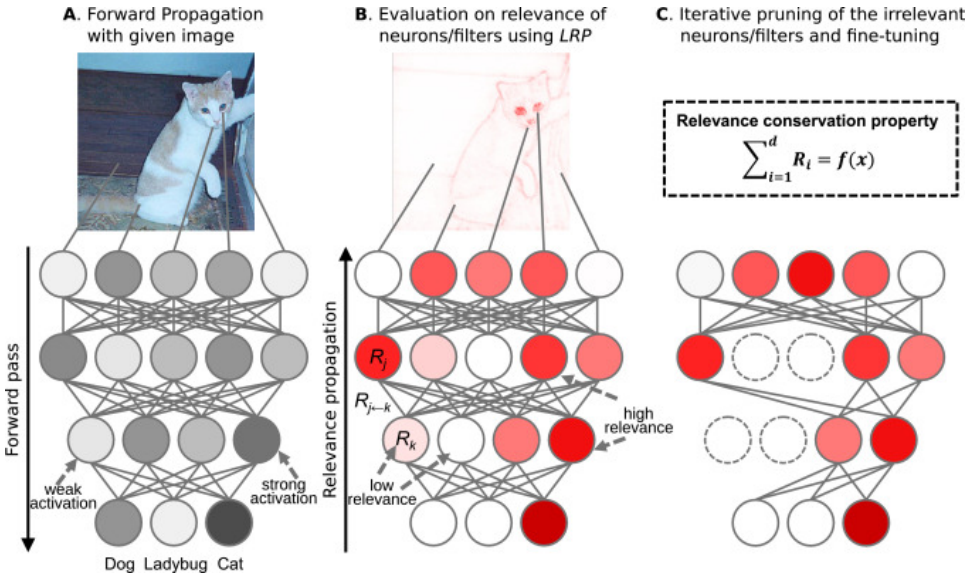

Abbildung 1: Vorgehen des LRP-Prunings A: Propagierung einer Eingabe (beispielhaft, das Bild einer Katze), B: Bemessung der Relevanzen von Gewichten mittels LRP, C: Iteratives Pruning der resultierenden irrelevantesten Gewichte, adaptiert aus [4]

Für das Pruning benötigt es hierbei vorerst eine Netzwerkpropagierung des Inputs, der entsprechend klassifiziert wird (in der Abbildung links zu sehen). Auf Grundlage des Resultats werden die Relevanzen den einzelnen Netzwerkeinheiten zugeordnet (in der Abbildung 1 mittig), um final irrelevante Parameter (siehe in der Abbildung rechts) iterativ zu entfernen. Um dabei die Relevanzen erhalten zu können, wird der Algorithmus mit $R_c^{(l)} = 1$ am Output initialisiert, wodurch dieser auch gegenüber eventuellen Unsicherheiten des Modells hinsichtlich Prädiktionen für Klassen robust bleibt.

B. Erklärbare Prototypen

Dem gegenüber stehen die von Chen et al. [5] präsentierten Prototypen. Um Klassifizierungen und Begründungen neuronaler Netzwerke transparenter zu gestalten, haben sie einen Ansatz entwickelt, der Konzepte innerhalb von Klassen ausfindig macht. Anhand von Labeln auf Bildebene - ohne explizite

Annotationen - erlernt ihr Framework visuelle Bestandteile, die maßgebend zur Prädiktion einer Klasse beitragen. “Prototypical Parts” innerhalb der Klassen schleusen so ein Level an Erklärbarkeit in ein Modell.

Ihr Netzwerk besteht hierbei aus einem Convolutional Neural Network, welches im Anschluss an die Konvolutionen ein Prototypen-Layer g_p besitzt. Die Anzahl an Klassen wird dem ProtoPNet lediglich über einen Parameter k übergeben. Jeder Prototyp repräsentiert hier ein Muster an Aktivierungen im Output der Konvolutionen, die einem Bildausschnitt des ursprünglichen Pixelraumes entsprechen. Über ein fully connected Layer h werden schließlich einzelne Prototypen den Klassen zugeordnet. Die Abbildung 2 verdeutlicht diese Struktur.

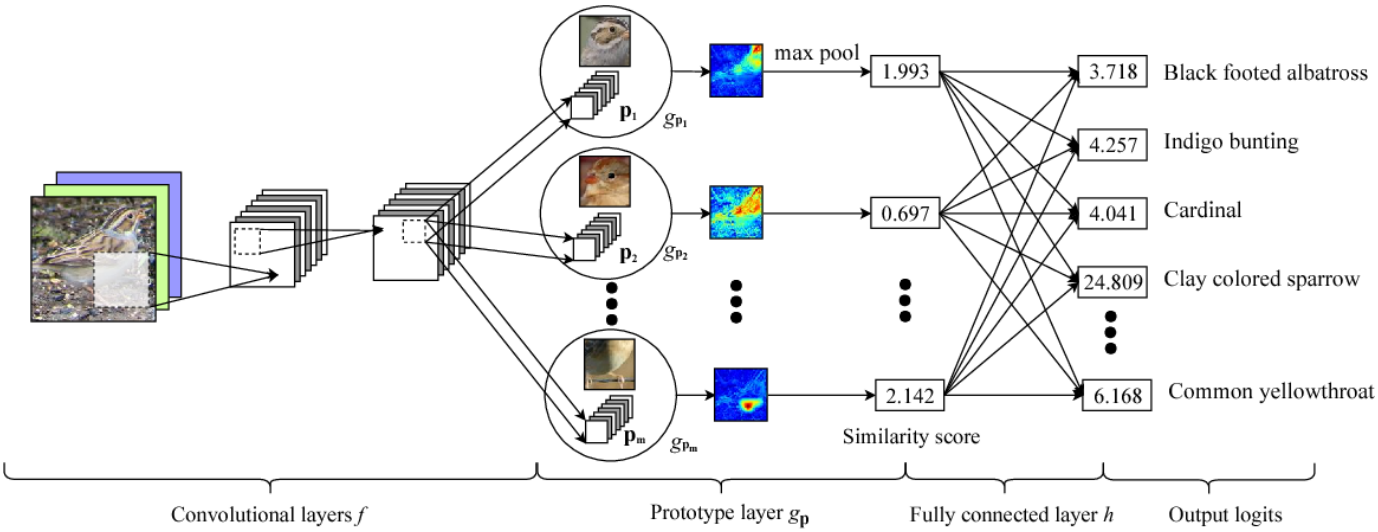

Abbildung 2: Architektur des ProtoPNets, adaptiert aus [5]

In dieser Hinsicht beschreiben die einzelnen Prototypen p_j die zugehörigen Klassen jeweils partiell, indem Distanzen zwischen den Prototypen und den Output-Patches berechnet werden. Die relevantesten Prototypen für die Klassen werden als m_k zusammengefasst. Um Prototypen zu erlernen, werden diese während des Trainings auf die naheliegendsten Training Patches der selben Klassen projiziert. Desweiteren wird im Layer h eine Optimierung angegangen, die die Unterscheidung und Zuordnung der Konzepte zu ihren jeweiligen Klassen bestärkt. Um Erlerntes zu visualisieren, werden die Heatmaps der Aktivierungen der einzelnen Einheiten p_j auf Training Samples berechnet.

III. Vorbereitung

Um das Pruning entsprechend umzusetzen, benötigt es vorerst eine Aufbereitung des Datensatzes, sodass beide Ansätze anschließend angewandt werden können. Im Rahmen dieser Arbeit wurde der HAM10000 Datensatz [6] verwendet. Dieser enthält zwei Ordner mit 10.000 Bildern zur Hautkrebsklassifikation, die von einem medizinischen Fachpersonal annotiert und in sieben verschiedene Klassen unterteilt wurden. Über die ebenfalls vorliegende CSV-Datei `HAM10000_metadata` wurden für die einzelnen Bilder die Klassensymbole `dx` anhand ihrer Namen ermittelt. Die von Experten vorgegebenen Klassen werden daraufhin mittels eines `lesion_type_dict` mit den durch die CSV-Datei vorgegebenen Kennzeichnungen für jedes Sample in Verbindung gebracht, wie in Algorithmus 1 zu sehen ist.

```
lesion_type_dict = {
    'nv': 'Melanocytic nevi',
    'mel': 'dermatofibroma',
    'bkl': 'Benign keratosis-like lesions ',
    'bcc': 'Basal cell carcinoma',
    'akiec': 'Actinic keratoses',
    'vasc': 'Vascular lesions',
    'df': 'Dermatofibroma'}
```

Algorithmus 1: Definition eines Dictionaries für die Rückwärtssuche

Für alle Bilder wird daraufhin arithmetisch der Mittelwert und die Standardabweichung ermittelt (Algorithmus 2):

```
for i in tqdm(range(len(image_paths))):
    img = cv2.imread(image_paths[i])
    img = cv2.resize(img, (img_h, img_w))
    imgs.append(img)

imgs = np.stack(imgs, axis=3)
imgs = imgs.astype(np.float32) / 255.

for i in range(3):
    pixels = imgs[:, :, i, :].ravel()
    means.append(np.mean(pixels))
    stdevs.append(np.std(pixels))
```

Algorithmus 2: Ermittlung der Mittelwerte und Standardabweichungen über alle Bilder

Über alle Bilder und ihre Pfade iterierend, wird mittels der Python-Bibliothek `Pandas` ein `Dataframe` erstellt, welcher ihre jeweiligen Pfade und Klassen enthält. Etwaige Duplikate werden entfernt und der Datensatz in ein Trainings- und Testdatensatz unterteilt.

Weiterführend wird für das Prototypen-basierte Pruning auch ein Validationsdatensatz erstellt, um das Modell zum Training zu überprüfen.

Mögliche Dysbalancen der Klassen durch eine unterschiedliche Anzahl an Samples des Trainingsdatensatzes werden durch ein Resampling anhand einer festgelegten Augmentationsrate gelöst. Dies wird im Algorithmus 3 veranschaulicht.

```
data_aug_rate = [15,10,5,50,0,40,5]
for i in range(7):
    if data_aug_rate[i]:
        df_train=df_train.append([df_train.loc[df_train['cell_type_idx'] == i,:]]*(
            data_aug_rate[i]-1), ignore_index=True)
```

Algorithmus 3: Resampling von unterrepräsentierten Klassen

Mittels `torch.transforms` wird eine Transformation erstellt, die auf dem Trainingsdatensatz angewandt die Bilder augmentiert (siehe Algorithmus 4):

```
train_transform = transforms.Compose([
                    transforms.Resize((224,224)),
                    transforms.RandomHorizontalFlip(),
                    transforms.RandomVerticalFlip(),
                    transforms.RandomRotation(20),
                    transforms.ColorJitter(brightness=0.1, contrast=0.1, hue=0.1),
                    transforms.ToTensor()
                    ])
```

Algorithmus 4: Definition der Transformation

Die Klasse `HAM10000` bildet über den von Pytorch implementierten Helfer `torch.utils.data.Dataset` einen individuellen, iterierbaren Datensatz. Der hier als Parameter übergebene `Dataframe` kann so iterativ für jeden Index anhand der jeweiligen Zellen einen Input `x` und einen Output `y` generieren (Algorithmus 5).

```
class HAM10000(Dataset):
 def __init__(self, df, transform=None):
     self.df = df
     self.transform = transform

 def __len__(self):
     return len(self.df)

 def __getitem__(self, index):
     x = Image.open(self.df['path'][index])
     y = torch.tensor(int(self.df['cell_type_idx'][index]))

     if self.transform:
         x = self.transform(X)

     return x, y
```

Algorithmus 5: HAM10000 Klasse als Wrapper für den Datensatz

Die Transformation des Datensatzes bleibt hierbei optional. Über den ebenfalls durch den Helfer implementierten `DataLoader` werden die einzelnen Datensätze für das anschließende Training und die Evaluation in iterierbare Wrapper eingebunden.

```
train_loader = torch.utils.data.DataLoader(training_set, batch_size=32, shuffle=True,
    num_workers=4)
val_loader = torch.utils.data.DataLoader(validation_set, batch_size=32, shuffle=False
   , num_workers=4)
test_loader = torch.utils.data.DataLoader(test_set, batch_size=32, shuffle=False,
   num_workers=4)
```

Algorithmus 6: Erstellung von Datensätzen anhand des DataLoader Wrappers

IV. Umsetzung

Den Kern dieser Arbeit bildend, werden zwei Methodiken untersucht und angewandt: Das Pruning von irrelevanten Filtern, welches mittels LRP geschieht, als auch das Pruning von Prototypen - dieses beruht auf ProtoPNet. Beide Ansätze werden weiterführend beschrieben.

A. *Pruning von irrelevanten Filtern*

Die im Unterkapitel Erklärbare Relevanzen vorgestellte, LRP-basierte Methode zum Finden von unbedeutenden Parametern wurde von Yeom et al. auf Github als Open Source Framework veröffentlicht, um speziell Filter eines Netzwerks zu prunen [7]. Dieses wurde im Rahmen dieser Arbeit bezüglich dieses Anwendungsfalls angepasst und kann unter folgendem Git-Repository gefunden werden: `https://github.com/ilonae/LRP_pruning_ICW1`.

Um auch auf dem HAM10000 Datensatz irrelevante Filter zu identifizieren, wurde das Modul `data` erweitert, um über `get_ham10000()` diesen wählen zu können.

Über optionale Argumente dieser Implementierung kann die Architektur gewählt werden. Da hier das ResNet18 verwendet wird, muss das `network`-Modul hinsichtlich der Eingaben angepasst werden. Um auch Eingaben der Größe 224 x 224 Pixel zu verarbeiten, werden Kernel- Stride- und Paddinggrößen des ersten Convolutional Layers angepasst. Auch das ReLU- und Max- und Average-Pooling Layer, welches üblicherweise in dieser Architektur vorhanden sind, fehlen für die korrekte Verarbeitung und wurden entsprechend eingefügt.

```
class ResNet(nn.Module):

    def __init__(self, block, num_blocks, num_classes=10):
        super(ResNet, self).__init__()

        self.in_planes = 64
        self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        self.layer1 = self._make_layer(block, 64, num_blocks[0], stride=1)
        self.layer2 = self._make_layer(block, 128, num_blocks[1], stride=2)
        self.layer3 = self._make_layer(block, 256, num_blocks[2], stride=2)
        self.layer4 = self._make_layer(block, 512, num_blocks[3], stride=2)
        self.avgpool = nn.AvgPool2d(7)
        self.linear = nn.Linear(512*block.expansion, num_classes)
```

Algorithmus 7: Anpassen der ResNet-Klasse

Für die durch Pytorch vorgegebene Propagierung werden `Hooks` implementiert, die über alle Layer jeweils ihre Outputs sammeln und weitergeben. Dies musste entsprechend der zusätzlichen Layer angepasst werden (siehe Algorithmus 8).

```
def forward(self, x):

    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)
    out = self.maxpool(out)
```

```
    out = self.layer1(out)
    out = self.layer2(out)
    out = self.layer3(out)
    out = self.layer4(out)

    out = self.avgpool(out)
    out = out.view(out.size(0), -1)

    out = self.linear(out)
    return out
```

Algorithmus 8: Anpassen des Forward Passes

Da für das letzte Layer für die Klassifizierung die korrekte Anzahl an Neuronen relevant sind, wurde der optionale Parameter `num_classes` hinzugefügt, der ebenfalls über Argumente eingegeben werden kann.

Auch das Modell `resnet_kuangliu` wurde entsprechend der geänderten Struktur angepasst. Dieses Modul dient als Wrapper für unterschiedliche ResNets, um durch das Modul `filter_prune` Hooks für den Forward und Backward Pass einzubinden und so Relevanzen der Filter ermitteln zu können.

Die Anzahl der zu prunenden Filter wird über das Argument für die Pruning-Rate gesetzt und das Ranking über die einzelnen Layer anhand des individuell wählbaren Pruning-Kriteriums (in diesem Fall LRP) gesammelt. Hierbei sammeln die Backward Hooks den Output und geben ihn durch einen Pass an das vorhergehende Layer zurück.

B. Pruning von Prototypen

Weiterführend wurde für die Umsetzung des Prototypen-basierten Prunings das ProtoPNet [8] abgewandelt. Die Anzahl an Klassen, die in den `settings` definiert ist, wurde auf 7 entsprechend des HAM10000-Datensatzes angepasst.

Da wie bereits in anderer Literatur [9] angemerkt auch die Wahl der Prototypen für jede Klasse spezifisch bedacht werden sollte, wurde die Anzahl an Prototypen auf 10 je Klassen beschränkt. Auch die Dimensionen des Outputs wurden abgeändert, entsprechend der Bildgröße (die bei dem HAM10000-Datensatz 224 x 224 Pixel sind).

Da es zu Schwierigkeiten bei der Speicherallokation kam, wurden desweiteren die Batchgrößen und die Anzahl an Epochen angepasst. Final wurde ebenso das ResNet18 als Basisstruktur verwendet. Entsprechend des Modells werden die festgelegten Features in `model` herausgefiltert, nach denen die Filtergrößen, Strides und Paddings im konvolutionellen Teil des Netzwerks festgelegt werden. Die Anwendung kann in dem Git-Repository unter `https://github.com/ilonae/ProtoPNet_ICW1` gefunden werden.

V. Evaluation

Dieses Kapitel zeigt die durch die Anwendung beider Ansätze erzielten Resultate auf und stellt sie gegen einander. Das Modell wurde in beiden Fällen über 10 Epochen trainiert und eine Test-Präzision von 90,21% erzielt.

Unter Zuhilfenahme der LRP-Implementierung wurden daraufhin Filter über das LRP- und das Gewichtsmagnitüden-Kriterium mit einer Pruning-Rate von 20% entfernt.

Innerhalb von 30 Pruning-Schritten wurde mittels beider Kriterien die Anzahl an Floating Point Operations (FLOPs) des ResNets von 1,8 auf 1,1 Billionen verringert. Die Abbildung 3 veranschaulicht den Verlauf.

Bei dem Pruning mittels LRP und anschließendem Fine-Tuning nahm die Test-Präzision von 61% bis 84% zu. Dem gegenüberstehend zeigt die Auswertung des Gewichtsmagnitüden-Kriteriums, dass die Wiederherstellung der Genauigkeit des Netzwerks sich inkonstanter verhält.

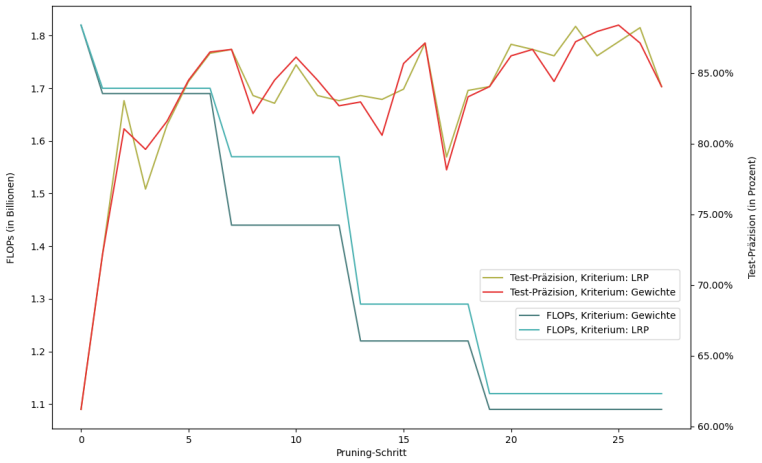

Abbildung 3: Auswertung von durch Gewichtsmagnitüden- und LRP-Pruning eingesparten FLOPs und durch Fine-Tuning verbesserte Präzisionen

Für das Prototypen-basierte Pruning wurden gesammelte Prototypen anhand von Samples visuell untersucht. Wie in der Abbildung 4 zu sehen ist, konnten für die Samples aus dem Validierungsdatensatz (obere Reihe) mittels der auf den Pixelraum gemappten Aktivierungen (mittlere Reihe) tatsächlich Konzepte innerhalb der Klassen gefunden werden, die diese näher beschreiben. In der unteren Reihe sind die zugeschnittenen Prototypen bezüglich ihrer Klassen zu sehen.

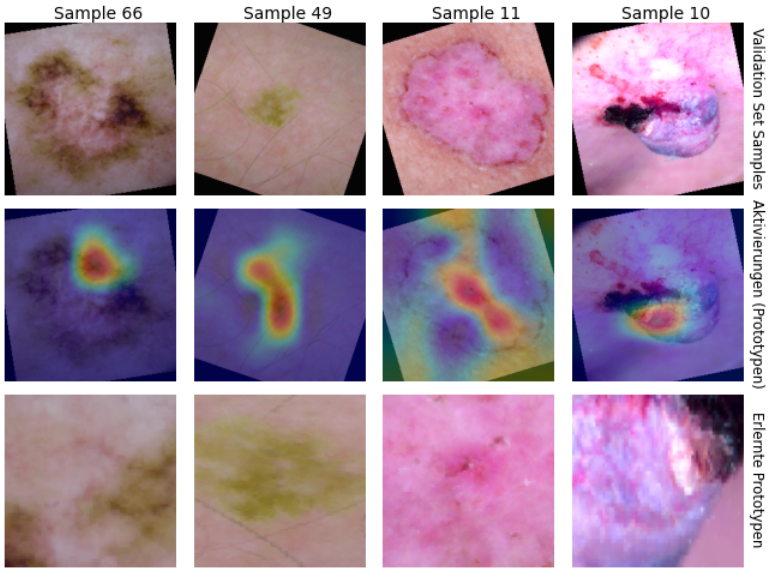

Abbildung 4: Mittels ProtoPNet erlernten Prototypen, obere Reihe: Beispielhafte Samples aus dem Validierungsdatensatz, mittlere Reihe: Aktivierungs-Heatmaps der Prototypen (klassenspezifisch) zu jedem Sample, untere Reihe: Zugeschnittene, erlernte Prototypen zu jedem Sample auf Grundlage der Klassen

Ebenso konnte für die geprunten Prototypen verfahren werden. In der oberen Reihe der Abbildung 5 sind hier Samples aus dem Validierungsdatensatz zu sehen, die mittig zu sehen Aktivierungen auf dem gemappten Pixelraum erzeugen.

Wie erkennbar wird, sind die Bildausschnitte nicht aussagekräftig genug für die Klassen und deuten teils

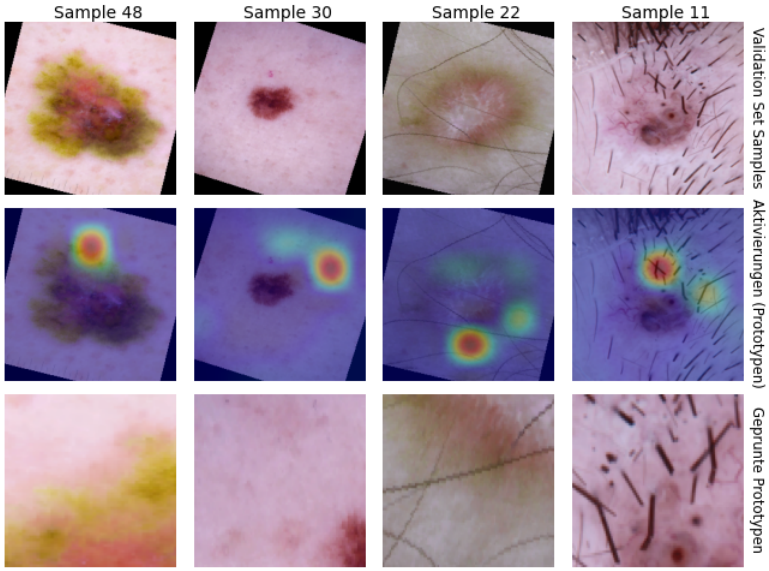

Abbildung 5: Mittels ProtoPNet geprunte Prototypen, obere Reihe: Beispielhafte Samples aus dem Validierungsdatensatz, mittlere Reihe: Aktivierungs-Heatmaps der Prototypen (klassenspezifisch) zu jedem Sample, untere Reihe: Zugeschnittene, geprunte Prototypen zu jedem Sample (diese sind nicht repräsentativ genug für die Klassen)

auch auf gesundes Gewebe hin. Da dies nicht zielführend genug für die genaue Prädiktion der Läsionen ist, können diese entsprechend gepruned werden. Daraus resultierend zeigt die untere Reihe die zugeschnittenen Prototypen, die aus dem Netzwerk entfernt werden.

Weiterführend wurden beide Ansätze verglichen und hier bezüglich ihrer Präzision in Abbildung 6 gegeneinander gestellt.

Pruning-Kriterium	**Präzision nach dem Pruning**	**Präzision nach anschließendem Fine-Tuning**
Aktivierungen (LRP)	72,16%	84,04%
Prototypen (ProtoPNet)	81,32%	90,11%
Magnitüden der Gewichte	72,16%	84,04%

Abbildung 6: Vergleich der Prädiktionspräzision nach dem Pruning und anschließendem Fine-Tuning der verschiedenen Ansätze

Nach dem Pruning zeigte das Netzwerk sowohl über LRP als auch über Gewichtsmagnitüden gehend starke Einbußen und kam zu einer Präzision von 72,16%. Auch das ProtoPNet musste mit 81,32% vorerst eine Verschlechterung verzeichnen. Durch anschließendes Fine-Tuning jedoch konnte für die verschiedenen Kriterien eine Verbesserung erzielt werden, weshalb das durch LRP-geprunte Netzwerk schließlich 84% und durch Prototypen-Pruning 90,11% erreichen konnte.

VI. Fazit

Final kann rückblickend ein Resümee zu dieser Arbeit aufgestellt werden. Hier konnte gezeigt werden, dass sowohl durch Prototypen als visuelle Erklärbarkeit, als auch LRP als Relevanzkriterium ein Pruning auf individuellen Datensätzen und Architekturen ermöglicht werden kann. Obwohl das Training und die Auswertung des Prunings viel Zeit in Anspruch nahmen und daher lediglich das ResNet18 mit einer Pruning-Rate von 20% ausgewertet wurde, konnte über LRP-Pruning die Netzwerkgröße und daraus resultierend die Anzahl an FLOPs verringert werden.

Durch ProtoPNet konnten außerdem tatsächlich adäquate Prototypen gefunden werden, die aussagekräftig zum Klassifizieren von Samples beitragen. Das Pruning unzureichend spezifischer Konzepte kann so zur genaueren Distinktion der Klassen führen.

Desweiteren kann das Adjustieren vorhandener Algorithmen bezüglich spezifischer Anwendungsfälle zielführend sein, um auch andere Architekturen, Datensätze und Methodiken praktisch zu realisieren.

Da das Training und das entsprechende Pruning viel Rechenleistung in Anspruch nehmen, wäre es sinnvoll, zukünftig mit stärkerer Hardware zu arbeiten. So könnte ein längeres Training und verschiedene Pruning-Raten gegen einander zu verglichen werden.

Da der LRP-Ansatz auch andere Kriterien mit einbindet, wäre es in diesem Rahmen auch interessant, hinsichtlich anderer Erklärbarkeiten - wie beispielsweise LIME [10] oder TCAV [11] - die Implementierung zu erweitern und auszuwerten.

LITERATUR

[1] S. Han, J. Pool, J. Tran und W. J. Dally, „Learning both Weights and Connections for Efficient Neural Networks," *CoRR*, Jg. abs/1506.02626, 2015. [Online]. Adresse: http://arxiv.org/abs/1506.02626.

[2] P. Molchanov, A. Mallya, S. Tyree, I. Frosio und J. Kautz, „Importance estimation for neural network pruning," Bd. abs/1906.10771, 2019. [Online]. Adresse: http://arxiv.org/abs/1906.10771.

[3] N. Lee, T. Ajanthan und P. H. S. Torr, „SNIP: Single-shot Network Pruning based on Connection Sensitivity," *ArXiv*, Jg. abs/1810.02340, 2019. [Online]. Adresse: https://arxiv.org/abs/1810.02340.

[4] S. Yeom, P. Seegerer, S. Lapuschkin, S. Wiedemann, K. Müller und W. Samek, „Pruning by Explaining: A Novel Criterion for Deep Neural Network Pruning," *CoRR*, 2019. [Online]. Adresse: http://arxiv.org/abs/1912.08881.

[5] C. Chen, O. Li, A. Barnett, J. Su und C. Rudin, „This looks like that: deep learning for interpretable image recognition," *CoRR*, 2018. [Online]. Adresse: http://arxiv.org/abs/1806.10574.

[6] P. Tschandl, C. Rosendahl und H. Kittler, „The HAM10000 Dataset: A Large Collection of Multi-Source Dermatoscopic Images of Common Pigmented Skin Lesions," *CoRR*, Jg. abs/1803.10417, 2018. arXiv: 1803.10417. [Online]. Adresse: http://arxiv.org/abs/1803.10417.

[7] S.-K. Yeom, P. Seegerer, S. Lapuschkin *et al.*, *Pruning by explaining: A novel criterion for deep neural network pruning (Github Repository)*, 2021. [Online]. Adresse: https://github.com/seulkiyeom/LRP_pruning.

[8] C. Chen, O. Li, C. Tao, A. J. Barnett und J. Su, *This looks like that - ProtoPNet (Github Repository)*, 2018. [Online]. Adresse: https://github.com/cfchen-duke/ProtoPNet.

[9] S. Mohammadjafari, M. Cevik, M. Thanabalasingam, A. Basar und A. D. N. Initiative, „Using ProtoPNet for Interpretable Alzheimer's Disease Classification," *Proceedings of the Canadian Conference on Artificial Intelligence*, 8. Juni 2021, https://caiac.pubpub.org/pub/klwhoig4. DOI: 10.21428/594757db.fb59ce6c. [Online]. Adresse: https://caiac.pubpub.org/pub/klwhoig4.

[10] M. T. Ribeiro, S. Singh und C. Guestrin, „"Why Should I Trust You?": Explaining the Predictions of Any Classifier," *CoRR*, Jg. abs/1602.04938, 2016. arXiv: 1602.04938. [Online]. Adresse: http://arxiv.org/abs/1602.04938.

[11] B. Kim, M. Wattenberg, J. Gilmer *et al.*, „Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)," 2017. [Online]. Adresse: https://arxiv.org/abs/1711.11279.